

WIPRO PYTHON AUTOMATION COURSE

MODULE 3

PYTHON BDD RESTFUL AUTOMATIONS

Training & Placement Preparation Report

Name: - Amit Raj

Course: - Python Automation Testing

Technology: - Python, Requests, Behave, REST APIs

Enrolment No.: - 12023052018002

Stream: - CSBS

Semester: - 7th

Date: - 21/09/2026

Abstract

Python BDD RESTful Automation involves API testing, python automation, and behavior driven development. RESTful APIs offer the functionality of an application by way of HTTP requests such as GET, POST, PUT, PATCH and DELETE. The Python Request library offers a practical means of making these requests and analyzing the response.

In the module, we start from basics of REST/SOAP and Requests to configuration, reusable payload, authentication, sessions, cookies, MySQL backed data, CRUD operations, response handling/error handling and finally Behavior Driven Development using Behave. This report offers you theory, practical example, framework design.

Table of Contents

1. Introduction to Python BDD RESTful Automation
 2. Module Scope and Learning Objectives
 3. 19.1 Python Basics
 4. 19.2 Python HTTP Libraries
 5. 19.3 API Automation
 6. 19.4 Handling HTTP Responses
 7. 19.5 Behavior-Driven Development
 8. Framework Architecture
 9. Configuration, Test Data and Database Strategy
 10. Best Practices and Common Mistakes
 11. Conclusion
- Appendix A – Commands

1. Introduction to Python BDD RESTful Automation

API Automation checks the backend services directly via HTTP instead of the GUI. RESTful APIs usually deal in JSON and use HTTP methods to expose resources. The API can be checked for its status codes, headers, body responses, business logic, security, persistence, and services.

Business-readable layer is provided by BDD. The feature files define the behavior in terms of "Given," "When" and "Then," while Python step definitions implement this behavior. Behave is an example of a Python BDD framework.

1.1 Why API Automation Matters

- Quick feedback in relation to many UI tests.
- Direct verification of service contracts and business logic.
- Easy positive, negative and boundary testing.
- Good for data driven regression suite.
- Perfect for CI/CD since browser does not need to run.

Ideal for microservices and backend integration.

1.2 REST vs SOAP

Aspect	REST	SOAP
Style	Architectural style using resources and HTTP	Protocol with structured XML messaging
Typical format	JSON, XML, text, etc.	XML
Operations	GET, POST, PUT, PATCH, DELETE	Service operations/messages
Contract	Often OpenAPI or other API documentation	Often WSDL-based
Common use	Web/mobile services and microservices	Enterprise integrations requiring SOAP standards
Testing focus	HTTP, JSON, headers, schema, business rules	XML envelope, WSDL, SOAP faults

1.3 HTTP Methods

Method	Typical purpose	Automation validation
GET	Retrieve data	Status, headers, response fields
POST	Create/submit	Creation status, ID, response body
PUT	Replace/update	Full update behavior
PATCH	Partial update	Only intended fields change
DELETE	Remove resource	Deletion status and subsequent state

1.4 API Test Flow

- Set the environment.
- Authentication and headers setup.
- Setup request payload/parameters.
- Make the request.
- Check response status and headers.
- Parse and check response.
- Save IDs/status for next requests.
- Clean up test data.
- Create reports and fail proof.

2. Module Scope and Learning Objectives

Section	Topics visible in supplied syllabus	Expected competency
19.1 Python Basics	REST vs SOAP; Python Requests Library; Python + Behave setup	Understand REST/API basics and set up the Python BDD environment
19.2 Python HTTP Libraries	GET calls; JSON response; Requests features; installation; status/header validation; POST	Automate HTTP requests and validate responses

	payload/headers; end-to-end API flow	
19.3 API Automation	Global config; external payloads; authentication; sessions; cookies; MySQL; CRUD; PUT/PATCH	Build reusable, realistic API automation
19.4 Handling HTTP Responses	Response objects/content; binary/text; status/error codes; exceptions; advanced handling; streaming; cookies/sessions	Handle normal and exceptional API responses
19.5 BDD	BDD tools; feature files; step definitions; hooks; parameterization; tagging; reusable code; failure reports	Build a maintainable Python BDD API framework

2.1 Prerequisites

- Python functions, dictionaries, lists and exceptions.
- Basic OOP.
- HTTP, JSON, headers and status codes.
- pip and command line.
- Basic SQL/MySQL.
- Testing and assertion fundamentals.

3. 19.1 Python Basics

3.1 Python Requests Library

Requests is a popular Python HTTP client library that simplifies GET, POST, PUT, PATCH and DELETE calls.

pip install requests

import requests

response = requests.get("https://api.example.com/users", timeout=10)

print(response.status_code)

print(response.headers)

print(response.text)

3.2 Query Parameters and JSON

```
params = {"page": 2, "limit": 10}
response = requests.get(
    "https://api.example.com/users",
    params=params,
    timeout=10
)
```

```
data = response.json()
```

3.3 Python + Behave Setup

```
python -m venv .venv
pip install requests behave mysql-connector-python
behave --version
```

Use a virtual environment to isolate dependencies. Record project dependencies in `requirements.txt`.

```
requests
behave
mysql-connector-python
```

4. 19.2 Python HTTP Libraries

4.1 GET HTTP Calls

```
import requests
```

```
url = "https://api.example.com/users/1"
response = requests.get(url, timeout=10)
```

```
assert response.status_code == 200
body = response.json()
print(body)
```

4.2 Basic Requests Features

Feature	Purpose
params	Query-string parameters
headers	Accept, Authorization and other headers
json	JSON request payload
data	Form/raw body data
cookies	Send cookies
auth	Supported authentication credentials
timeout	Limit wait time
Session	Reuse headers/cookies/connections

4.3 Validate Status Codes and Headers

```
response = requests.get(url, timeout=10)
```

```
assert response.status_code == 200
```

```
content_type = response.headers.get("Content-Type", "")
```

```
assert "application/json" in content_type
```

4.4 POST with Payload and Headers

```
payload = {"name": "Test User", "email": "test@example.com"}
```

```
headers = {
```

```
"Accept": "application/json",
```

```
"Content-Type": "application/json"
```

```
}
```

```
response = requests.post(
```

```
"https://api.example.com/users",
```

```
json=payload,
```

```
headers=headers,
```

```
timeout=10
```

```
)
```

```
assert response.status_code in (200, 201)
```

4.5 End-to-End API Flow

```
base = "https://api.example.com"
created = requests.post(
    f"{base}/users",
    json={"name": "Test User"},
    timeout=10
)
assert created.status_code in (200, 201)
user_id = created.json()["id"]
fetched = requests.get(f"{base}/users/{user_id}", timeout=10)
assert fetched.status_code == 200

updated = requests.patch(
    f"{base}/users/{user_id}",
    json={"name": "Updated User"},
    timeout=10
)
assert updated.status_code in (200, 204)
deleted = requests.delete(f"{base}/users/{user_id}", timeout=10)
assert deleted.status_code in (200, 202, 204)
```

The exact status codes and endpoint behavior must follow the API contract; these examples illustrate the automation pattern.

5. API Automation

5.1 Global Configuration Using a Python Config Object

Environment-specific values such as base URL and timeouts should be separated from test logic.

```
from dataclasses import dataclass
import os
@dataclass(frozen=True)
class Config:
    base_url: str = os.getenv("BASE_URL", "https://api.example.com")
    timeout: int = int(os.getenv("API_TIMEOUT", "10"))
config = Config()
```

Sensitive credentials should come from environment variables, CI secrets or an approved secret-management system.

5.2 External Reusable Payloads

```
# data/create_user.json
{
  "name": "Test User",
  "email": "test@example.com"
}import json
with open("data/create_user.json", encoding="utf-8") as f:
    payload = json.load(f)
    response = requests.post(
        f"{config.base_url}/users",
        json=payload,
        timeout=config.timeout
    )
```

- Separate test data from automation code.
- Keep secrets out of ordinary data files.
- Use reusable loaders for JSON/CSV data.
- Give datasets meaningful names.

5.3 API Authentication

Common API authentication mechanisms include Basic Authentication, API keys, bearer tokens and OAuth 2.0. The correct mechanism must be taken from the API contract.

```
# Basic authentication
response = requests.get(
    "https://api.example.com/profile",
    auth=("username", "password"),
    timeout=10
)

# Bearer token
headers = {
    "Authorization": f"Bearer {token}",
    "Accept": "application/json"
}
response = requests.get(url, headers=headers, timeout=10)
```

5.4 Session Management

`requests.Session()` persists session-level settings such as cookies and headers and is useful for sequences of related calls.

```
session = requests.Session()
session.headers.update({"Accept": "application/json"})
login = session.post(
    "https://api.example.com/login",
    json={"username": "demo", "password": "secret"},
    timeout=10
)
profile = session.get(
    "https://api.example.com/profile",
    timeout=10
)
```

5.5 Cookies

```
cookies = {"session_id": "example-session"}
response = requests.get(
    "https://api.example.com/profile",
    cookies=cookies,
    timeout=10
)
print(response.cookies)
```

A Session can also automatically retain cookies returned by the server between requests.

5.6 MySQL Integration

pip install mysql-connector-python

```
import mysql.connector
connection = mysql.connector.connect(
    host="localhost",
    user="test_user",
    password="test_password",
    database="automation_db"
)
cursor = connection.cursor()
cursor.execute("SELECT id, name FROM users")
```

```
rows = cursor.fetchall()
for row in rows:
    print(row)
cursor.close()
connection.close()
```

For real projects, credentials should be protected and SQL values should be parameterized.

5.7 Example Database Table

```
CREATE TABLE users (
id INT PRIMARY KEY AUTO_INCREMENT,
name VARCHAR(100) NOT NULL,
email VARCHAR(150) NOT NULL
);
INSERT INTO users (name, email)
VALUES ('Test User', 'test@example.com');
```

5.8 PUT, PATCH and DELETE

```
# PUT
response = requests.put(
f'{base_url}/users/10',
json={"name": "New Name", "email": "new@example.com"},
timeout=10
)

# PATCH
response = requests.patch(
f'{base_url}/users/10',
json={"name": "Only Name Changed"},
timeout=10
)

# DELETE
response = requests.delete(
f'{base_url}/users/10',
timeout=10
)
```

5.9 Updating and Deleting Data via API

- Create a uniquely identifiable test record.
- Capture its ID from the response.
- Retrieve and validate the initial record.
- Update using PUT or PATCH according to the contract.
- Retrieve again and validate the changed fields.
- Delete/restore the test record when appropriate.
- Verify the final state.

5.10 PUT vs PATCH

Aspect	PUT	PATCH
Typical intent	Replace/update a resource representation	Partially modify a resource
Payload	Often contains the complete representation required by the contract	Usually contains only changed fields/operations
Idempotency	Defined as idempotent by HTTP semantics	Depends on the specific operation
Test focus	Validate replacement/update semantics	Validate only targeted fields changed

6. Handling HTTP Responses

6.1 Understanding Response Objects

```
response = requests.get("https://api.example.com/users/1", timeout=10)
print(response.status_code)
print(response.headers)
print(response.url)
print(response.text)
print(response.content)
```

6.2 Reading Response Content

```
response = requests.get(url, timeout=10)
content_type = response.headers.get("Content-Type", "")
if content_type.startswith("application/json"):
    body = response.json()
else:
    body = response.text
```

6.3 Binary and Text Data

```
response = requests.get(file_url, timeout=20)
with open("downloaded_file.bin", "wb") as f:
    f.write(response.content)
```

response.content returns bytes;

response.text returns decoded text.

Use streaming for very large responses.

6.4 Status Codes and Error Handling

Status	Meaning	Typical test action
2xx	Successful request	Validate expected business result
3xx	Redirection	Validate redirect behavior if required
4xx	Client/auth/request problem	Validate negative scenario and error body
5xx	Server-side problem	Record evidence and classify failure
401	Authentication required/invalid	Validate token/credential behavior
403	Forbidden	Validate authorization behavior
404	Resource not found	Validate expected missing-resource behavior

409	Conflict in many APIs	Validate duplicate/state conflict
-----	-----------------------	-----------------------------------

6.5 response.ok vs Exact Assertions

```
response = requests.get(url, timeout=10)
assert response.ok
assert response.status_code == 200
```

provides a broad success check. For automation, exact expected status codes are usually more informative.

6.6 Exception Handling in Requests

```
try:
    response = requests.get(url, timeout=5)
    response.raise_for_status()
except requests.exceptions.Timeout:
    print("Request timed out")
except requests.exceptions.ConnectionError:
    print("Connection failed")
except requests.exceptions.HTTPError as exc:
    print("HTTP error:", exc)
```

A 404 is an HTTP response; a connection failure may occur before a response exists. Tests should distinguish these conditions.

6.7 Advanced Response Handling

- Validate Content-Type before parsing.
- Validate required JSON fields and nested structures.
- Use schema validation when the API contract requires it.
- Capture correlation/request IDs when provided.
- Set timeouts on requests.
- Redact sensitive response fields in reports.

6.8 Streaming Large Responses

```
with requests.get(large_file_url, stream=True, timeout=30) as response:
    response.raise_for_status()
    with open("large_file.dat", "wb") as output:
        for chunk in response.iter_content(chunk_size=8192):
```

```
if chunk:
    output.write(chunk)
```

6.9 Cookies and Sessions

```
session = requests.Session()
login = session.post(
    login_url,
    json={"username": "demo", "password": "secret"},
    timeout=10
)
profile = session.get(profile_url, timeout=10)
assert profile.status_code == 200
```

7. Behavior-Driven Development (BDD)

BDD describes application behavior through examples understandable by business and technical stakeholders. Behave is a Python BDD framework based on Gherkin syntax.

7.1 BDD Tools for Python

Tool	Purpose
Behave	Python BDD framework using Gherkin feature files
pytest-bdd	BDD layer integrated with PyTest
Robot Framework BDD-style keywords	Readable Given/When/Then convention without requiring Gherkin

The supplied syllabus specifically emphasizes Behave and environment.py, so Behave is the primary implementation here.

7.2 Feature File

Feature: User API

Scenario: Get an existing user

Given the API is available

When I send a GET request for user "1"

Then the response status should be 200

And the response should contain user id "1"

7.3 Step Definitions

```
from behave import given, when, then
import requests

@given("the API is available")
def step_api_available(context):
    context.base_url = "https://api.example.com"
    @when('I send a GET request for user "{user_id}"')
    def step_get_user(context, user_id):
        context.response = requests.get(
            f'{context.base_url}/users/{user_id}',
            timeout=10
        )
    @then("the response status should be {status:d}")
    def step_status(context, status):
        assert context.response.status_code == status
    @then('the response should contain user id "{user_id}"')
    def step_body(context, user_id):
        assert str(context.response.json()["id"]) == user_id
```

7.4 Hooks in environment.py

```
def before_all(context):
    context.base_url = "https://api.example.com"
    def before_scenario(context, scenario):
        context.response = None
    def after_scenario(context, scenario):
        if scenario.status == "failed" and context.response is not None:
            print("Response:", context.response.text)
    def after_all(context):
        pass
```

Hooks are appropriate for framework lifecycle tasks. Keep business logic in steps or service classes.

7.5 Scenario Outline / Parameterization

Feature: User API

Scenario Outline: Validate user retrieval

Given the API is available

When I send a GET request for user "<user_id>"
Then the response status should be <status>

Examples:

user_id	status
1	200
999999	404

7.6 Tagging

@smoke

Feature: User API

@positive

Scenario: Get an existing user

...

@negative

Scenario: Get an unknown user

...

behave --tags=smoke

behave --tags=negative

7.7 Reusable Step Definitions

- Keep steps at business-behavior level.
- Avoid duplicate steps with slightly different wording.
- Move HTTP mechanics into reusable API client classes.
- Centralize common assertions.
- Use context for scenario state carefully.
- Use tags consistently.

7.8 Generate Failure Reports

behave -f pretty

behave -f json -o reports/behave.json

8. End-to-End API Automation Framework Architecture

```
python_api_bdd/  
├── features/  
│   ├── users.feature  
│   ├── auth.feature  
│   └── environment.py  
├── steps/  
│   ├── user_steps.py  
│   └── auth_steps.py  
├── api/  
│   ├── client.py  
│   ├── users_api.py  
│   └── auth_api.py  
├── assertions/  
│   └── response_assertions.py  
├── config/  
│   └── settings.py  
├── data/  
│   ├── payloads/  
│   └── db/  
│       ├── mysql_client.py  
│       └── reports/  
└── requirements.txt
```

Layer	Responsibility
Feature files	Business-readable scenarios and expected behavior
Step definitions	Map Gherkin steps to Python operations
API client	Common HTTP/session/request behavior
Endpoint modules	Resource-specific API methods
Assertions	Reusable response validation
Configuration	Base URL, timeouts, environment settings

Data	External payloads and datasets
DB layer	Database setup/query/verification/cleanup
Reports	Execution results and failure evidence

8.1 Execution Flow

- Behave discovers selected feature files.
- environment.py initializes shared resources.
- Scenario steps call reusable API/client code.
- Requests sends the HTTP call.
- Assertions validate status/body/headers/business rules.
- Hooks perform cleanup or failure evidence.
- Behave produces local/CI output.

9. Configuration, Test Data and Database Strategy

9.1 Configuration Management

```
# settings.py
import os
BASE_URL = os.getenv("BASE_URL", "https://api.example.com")
API_TIMEOUT = int(os.getenv("API_TIMEOUT", "10"))
```

Configuration allows the same tests to run against different controlled environments without editing feature files.

9.2 External Payloads

- JSON is useful for structured API payloads.
- CSV is useful for simple tabular datasets.
- Database queries can provide dynamic/persistent data.
- Keep credentials separate from test data.

9.3 Database Verification Strategy

- Prepare or identify test data.
- Execute the API request.
- Validate HTTP response.
- Query the relevant database record with a parameterized query.
- Compare expected and actual state.

- Clean up data where appropriate.

9.4 Parameterized SQL

```
cursor.execute(
"SELECT id, name FROM users WHERE email = %s",
(email,)
)
```

Parameterized queries help avoid SQL injection risks and correctly handle values.

10. Reporting, Debugging and Failure Analysis

10.1 Failure Evidence

- Feature/scenario name.
- HTTP method and endpoint.
- Safe request parameters.
- Redacted headers.
- Status code.
- Relevant response headers/body.
- Exception details if no response exists.
- Correlation/request ID when available.

10.2 Failure Classification

Failure type	Example	Action
Test defect	Incorrect expected status	Fix test
Automation defect	Wrong endpoint/payload	Fix client or test data
Application defect	Valid request returns unexpected 5xx	Raise defect with evidence
Environment issue	Service unavailable	Verify environment and rerun appropriately
Data issue	Required record missing	Repair controlled data
Network issue	Timeout/connection error	Check connectivity/service

10.3 Safe Logging

Logs are valuable, but authorization headers, passwords, tokens, cookies and sensitive personal data should be redacted.

11. Best Practices and Common Mistakes

Common mistake	Better practice
Hard-coded base URL	Centralize configuration.
Credentials in source code	Use environment variables/CI secrets.
Only checking status code	Validate relevant body, headers and business rules.
No timeout	Set explicit request timeout.
Duplicated request logic	Use reusable API client.
All logic in step definitions	Move HTTP mechanics to service/client modules.
No cleanup	Create and remove controlled test data.
Secrets in logs	Redact sensitive values.
Overly technical BDD steps	Keep scenarios business-readable.
Environment-specific data mixed with code	Separate configuration, data and logic.

11.1 Assertion Strategy

```
response = client.post("/users", json=payload)
assert response.status_code == 201
assert response.headers["Content-Type"].startswith("application/json")
body = response.json()
assert "id" in body
assert body["name"] == payload["name"]
```

11.2 Idempotency and Repeatability

Tests should avoid depending on unrelated test data. Create unique records, capture IDs and clean them up. Understanding idempotency helps with safe retries and repeatable execution.

12. Conclusion

Module 3.0 provides a complete foundation for Python REST API automation with BDD. It begins with REST/SOAP concepts and Requests, progresses through HTTP methods and response validation, and introduces framework concerns such as configuration, external payloads, authentication, sessions, cookies, MySQL and CRUD workflows.

The BDD portion adds business-readable scenarios using Behave, feature files, step definitions, hooks, parameterization, tags and reusable code. For placement preparation, the learner should be able to build a small working API framework, execute it from the command line and explain.

Appendix A – Quick Command Sheet

Command	Purpose
<code>python -m venv .venv</code>	Create virtual environment.
<code>pip install requests</code>	Install Requests.
<code>pip install behave</code>	Install Behave.
<code>pip install mysql-connector-python</code>	Install MySQL connector.
<code>behave</code>	Run discovered features.
<code>behave --tags=smoke</code>	Run smoke-tagged scenarios.
<code>behave --tags=regression</code>	Run regression-tagged scenarios.
<code>behave -f pretty</code>	Readable console output.
<code>behave -f json -o reports/behave.json</code>	JSON execution output.